



**RÉPUBLIQUE
FRANÇAISE**

*Liberté
Égalité
Fraternité*



**UNIVERSITÉ
CAEN
NORMANDIE**

RAPPORT DE PROJET

Game Of Taquin

Fait par :

KODJO Afiwa Aimee

MILANDOU Promise Benite

Parcours : Licence 2 Informatique

Groupe : 3

Prof du TP :

Olivier Ranaivason

Table des matières

1	Introduction	2
2	Présentation claire des objectifs du projet	2
3	Organisation du Projet	2
3.1	Gestion du projet	2
3.2	Compilation du Projet	3
4	Architecture du projet	3
4.1	Arborescence du projet	3
4.2	Diagramme illustrant la Mise en place du pattern MVC	4
5	Aspects techniques	4
5.1	GrilleModele	4
5.1.1	Création de la grille	4
5.1.2	Mouvement des cases	5
5.1.3	Partie Terminée	6
6	Affichage du jeu	7
6.1	Spécificités et points durs du projet	7
7	Conclusion	8
8	Annexe	8

1 Introduction

L'apparition du jeu du taquin vers 1870 aux États-Unis a marqué le début d'une fascination pour les puzzles sous forme de damier [1]. Ce jeu, constitué de cases dont la disposition est modifiable, offre un défi captivant : reconstituer l'ordre initial à partir d'une configuration mélangée. Avec des cases affichant des lettres, des chiffres ou même des morceaux d'images, le joueur se lance dans un exercice de réflexion et de manipulation [2]. C'est dans ce contexte que s'inscrit notre projet, une application de jeu réalisée en Java, intégrant une interface graphique pour une expérience immersive. Toutefois, conscient de l'importance de l'accessibilité, nous avons également veillé à ce que l'application puisse être utilisée sans interface graphique. Pour ce faire, nous avons adopté le motif de conception MVC (Modèle-Vue-Contrôleur), offrant ainsi une structure robuste et modulaire à notre application.

2 Présentation claire des objectifs du projet

Le projet vise à développer une application de jeu basée sur le puzzle à glissières, le taquin, avec les points suivants :

- **Interface Graphique et Utilisation en Ligne de Commande :** L'application doit offrir une expérience de jeu à la fois avec une interface graphique et en ligne de commande, assurant ainsi une accessibilité flexible pour les utilisateurs.
- **Utilisation du MVC** L'architecture logicielle doit suivre le motif de conception Modèle-Vue-Contrôleur (MVC), garantissant ainsi une séparation claire entre les différentes composantes de l'application et une maintenabilité accrue du code.
- **Qualité et Robustesse du Code** Le code doit être de qualité, facile à comprendre et à maintenir, avec des tests effectués pour garantir sa robustesse. Cette approche assure une fiabilité et une évolutivité de l'application sur le long terme.

3 Organisation du Projet

3.1 Gestion du projet

Nous avons mis en place l'utilisation d'un gestionnaire de version afin de centraliser le code et d'améliorer l'efficacité du travail en équipe. Un dépôt a été créé sous Subversion, auquel ont été ajoutés les noms des deux membres

du groupe. De plus, Olivier Ranaivason a été ajouté comme managers du projet. Des efforts supplémentaires ont été consentis en dehors des heures de travaux pratiques pour mener à bien ce projet.

3.2 Compilation du Projet

Nous avons choisi d'utiliser Apache Ant, un logiciel de construction automatisée d'applications, qui se sert d'un fichier XML pour décrire les associations entre les commandes et les tâches à réaliser.

Pour exécuter les différentes commandes, nous utilisons l'exécution en ligne de commande :

- **ant** : pour compiler le projet.
- **ant run** : pour lancer le projet.
- **ant javadoc** : pour générer la Javadoc du projet.

Ce script est facilement exécutable grâce à Apache Ant, ce qui facilite le processus de compilation et de gestion du projet.

4 Architecture du projet

4.1 Arborescence du projet

La structure de notre projet est organisée de manière à faciliter la gestion du code source, des ressources et des tests. Il est composé de cinq principaux dossiers :

- **build** : Ce dossier, actuellement vide, est destiné à recevoir les fichiers ".class" après la compilation du code.
- **doc** : Ce dossier contient la documentation du projet.
- **vues** : Ce dossier contient l'ensemble des classes liées à la vues de l'application.
- **src** : C'est dans ce dossier que se trouve l'ensemble du code source de l'application.

En outre, d'autres fichiers sont présents à la racine du projet tels que :

- **build.xml** : Ce fichier est utilisé par Apache Ant pour automatiser la compilation, la création de la documentation, et d'autres tâches de construction du projet.
- **Diagramme** : Ce dossier contient un fichier au format ".dia" qui représente un diagramme lié au projet.

- **Rapport** : Ce fichier au format “.pdf” qui représente le rapport lié au projet.

4.2 Diagramme illustrant la Mise en place du pattern MVC

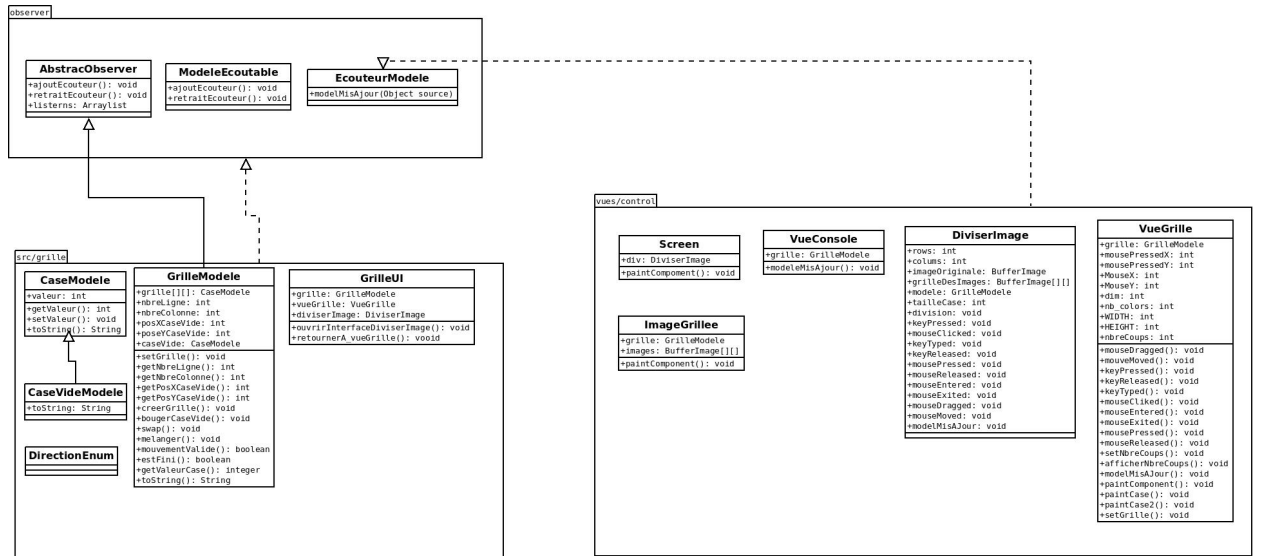


FIGURE 1 – Diagramme.

5 Aspects techniques

Concernant l’aspect technique plusieurs aspects peuvent être abordés, notamment La classe GrilleModele représente la grille du jeu du taquin. Elle initialise la grille, gère les mouvements des cases, assure le mélange aléatoire de la grille tout en maintenant sa jouabilité, vérifie si le jeu est terminé et fournit des méthodes utilitaires pour récupérer les valeurs des cases et afficher la grille.

5.1 GrilleModele

5.1.1 Création de la grille

Nous avons conçu la méthode creerGrille() pour initialiser la grille du jeu du taquin. Cette méthode parcourt chaque cellule de la grille et attribue une valeur à chaque case, en commençant par 1 jusqu’à ce que toutes les cases

soient remplies. Ensuite, une case est spécifiquement désignée comme vide pour représenter la position initiale du jeu.

Algorithme 1 : Méthode creerGrille()

Entrées : Taille de la grille

Output : Grille initialisée avec des cases ordonnées

```
1 pour chaque ligne dans grille faire
2   pour chaque colonne dans grille faire
3     grille [ligne][colonne]  $\leftarrow$  NouvelleCase(valeur);
4     valeur  $\leftarrow$  valeur +1;
5   fin
6 fin

7 DerniereLigne  $\leftarrow$  tailleGrille -1;
8 DerniereColonne  $\leftarrow$  tailleGrille -1;
9 grille [DerniereLigne][DerniereColonne]  $\leftarrow$  CaseVide;
10 retourner Grille;
```

5.1.2 Mouvement des cases

Pour permettre aux joueurs de déplacer les cases dans la grille, nous avons mis en place les méthodes bougerCaseVide() et swap(). La méthode bougerCaseVide() déplace la case vide dans une direction donnée, tandis que la méthode swap() échange la position de la case vide avec une case voisine. Ces méthodes garantissent que seuls les mouvements valides sont effectués, assurant ainsi l'intégrité de la grille du jeu

Algorithme 2 : Méthode bougerCaseVide(sens)

Entrées : Direction du mouvement (sens)

Output : Grille avec la case vide déplacée dans la direction spécifiée

```
1 suivant sens faire
2   cas où HAUT faire
3     si positionCaseVide [ligne]  $\neq 0$  alors
4       Échanger les positions entre la case vide et la case
        au-dessus;
5       positionCaseVide [ligne]  $\leftarrow$  positionCaseVide [ligne] - 1;
6     fin
7   fin
8   cas où BAS faire
9     si positionCaseVide [ligne]  $\neq$  tailleGrille- 1 alors
10      Échanger les positions entre la case vide et la case
        en-dessous;
11      positionCaseVide [ligne]  $\leftarrow$  positionCaseVide [ligne] + 1;
12    fin
13  fin
14  cas où GAUCHE faire
15    si positionCaseVide [colonne]  $\neq 0$  alors
16      Échanger les positions entre la case vide et la case à
        gauche;
17      positionCaseVide [colonne]  $\leftarrow$  positionCaseVide [colonne] -
        1;
18    fin
19  fin
20  cas où DROITE faire
21    si positionCaseVide [colonne]  $\neq$  tailleGrille- 1 alors
22      Échanger les positions entre la case vide et la case à droite;
23      positionCaseVide [colonne]  $\leftarrow$  positionCaseVide [colonne] +
        1;
24    fin
25  fin
26 fin
```

5.1.3 Partie Terminée

Nous avons une méthode `estFini` dans notre classe `Grille` responsable de déterminer si le jeu est terminé en vérifiant si toutes les cases sont correcte-

ment ordonnées, à l'exception de la case vide. l'importance de cette méthode offre une expérience de jeu satisfaisante aux utilisateurs en leur indiquant clairement quand ils ont résolu le puzzle avec succès.

6 Affichage du jeu

La présentation du jeu s'articule autour d'une grille 3x3, offrant une expérience fluide à travers une interface graphique intuitive et une alternative en mode console pour une accessibilité accrue. À chaque lancement de l'application, le jeu est mélangé dans une configuration qui garantit une solution réalisable. Une fonctionnalité visuelle remarquable est la mise en évidence des éléments déplaçables, signalée par une couleur verte lorsqu'ils sont survolés par la souris.

L'interface graphique propose une découpe et un mélange d'une image pour offrir une expérience visuelle immersive. En parallèle, une vue supplémentaire fournit des informations cruciales telles que le nombre de coups effectués, permettant ainsi aux joueurs de suivre leur progression et de mesurer leur performance.

6.1 Spécificités et points durs du projet

Dans la conception et le développement de notre projet, quelques défis significatifs ont été identifiés, mettant en lumière les aspects techniques et conceptuels qui ont nécessité une attention particulière. Parmi ces défis, le principal réside dans la mise en œuvre de la fonctionnalité permettant le déplacement des images dans la grille.

L'incapacité à déplacer les images, bien que désirée pour une expérience de jeu interactive, a été un point de difficulté majeur. Cette contrainte a nécessité une exploration approfondie des mécanismes de gestion des événements de souris et de clavier, ainsi qu'une analyse minutieuse des interactions entre les éléments graphiques de l'interface utilisateur.

La résolution de ce défi a demandé une approche ingénieuse, impliquant la révision des méthodes de découpe et de placement des images dans la grille, ainsi que la modification des mécanismes de détection des mouvements de souris pour les rendre compatibles avec les opérations de glissement des images.

En mettant en avant cette difficulté, notre rapport illustre l'engagement à relever les défis techniques tout en préservant la qualité et l'expérience utilisateur attendues dans notre projet.

7 Conclusion

En conclusion, ce projet nous a permis d'explorer en profondeur les concepts fondamentaux du jeu de taquin et de mettre en pratique nos compétences en programmation orientée objet. Dès le début, il a été facile de répartir les tâches, ce qui a permis à chaque membre de l'équipe de savoir clairement ce qu'il devait accomplir.

Nous avons réussi à développer le jeu du taquin, offrant à la fois une interface graphique conviviale et une utilisation en mode console. En utilisant le design pattern MVC, nous avons pu organiser notre code de manière claire et modulaire, facilitant ainsi sa maintenance et son évolution.

Ce projet nous a présenté un ensemble de défis techniques, notamment des problèmes avec la fonctionnalité de clic sur les cases de l'image découpée. Bien que nous ayons rencontré des difficultés avec l'interface graphique, nous avons réussi à surmonter ces obstacles grâce à une collaboration étroite et à des efforts soutenus.

En résumé, ce projet nous a permis d'acquérir de précieuses compétences en programmation, en conception logicielle et en gestion de projet.

Références

- [1] Google. <https://fr.wikipedia.org/wiki/Taquin>.
- [2] Wikipedia. Taquin (puzzle). *Wikipedia*, 2024.

8 Annexe

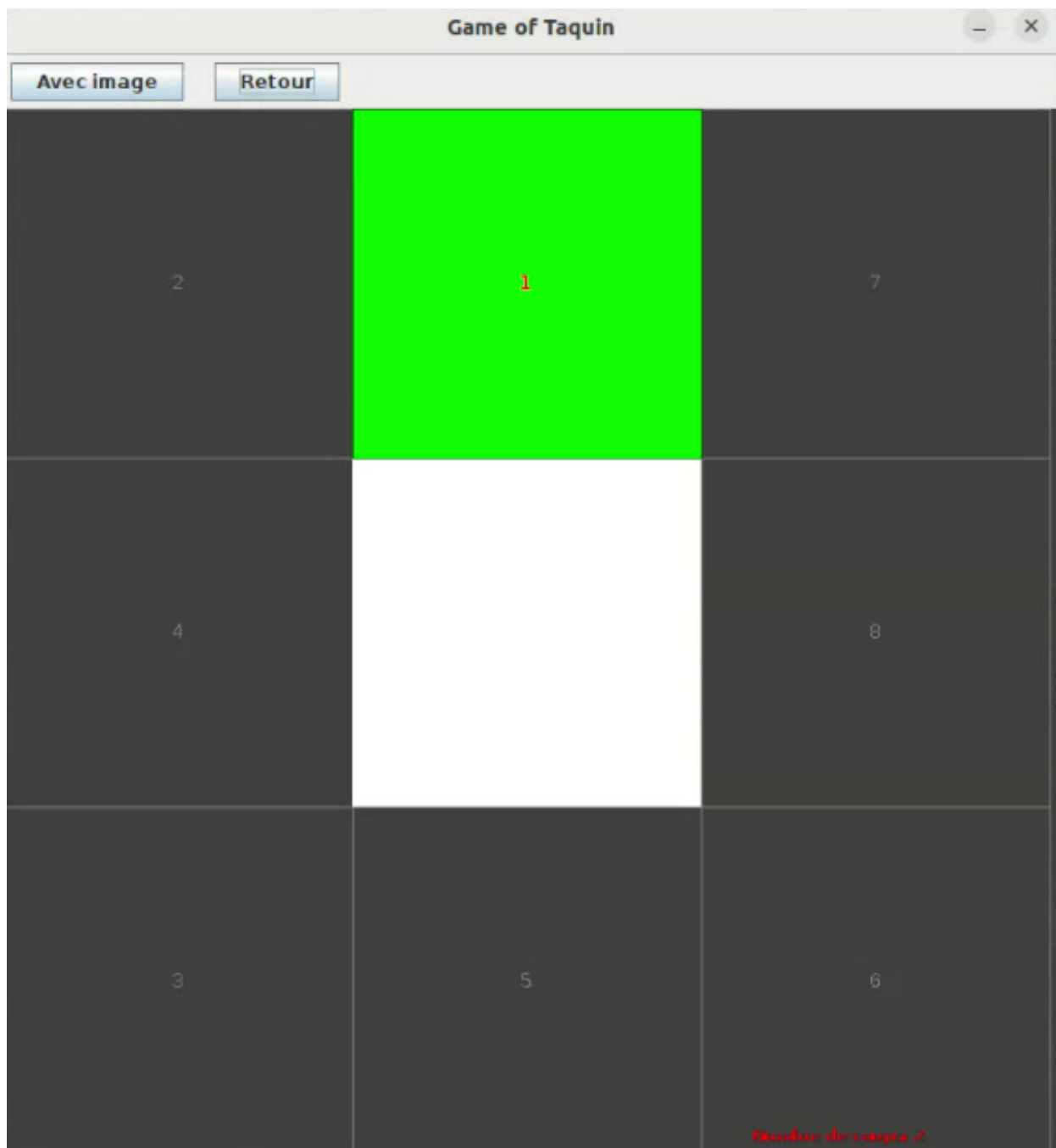


FIGURE 2 – Affichage

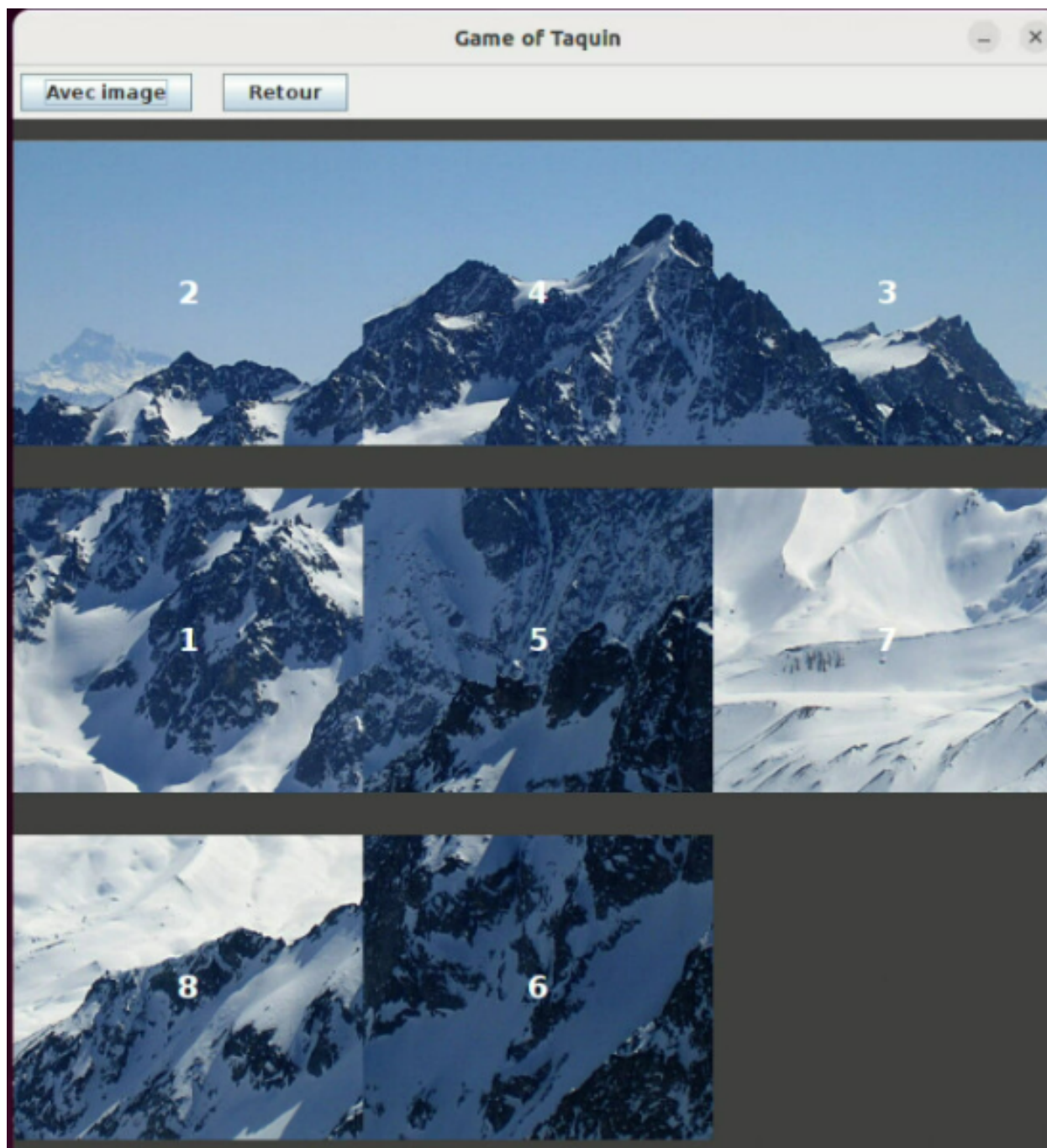


FIGURE 3 – Affichage avec Image

1	2	3
4	5	6
7	0	8

1	2	3
4	5	6
7	0	8

1	2	3
4	5	6
0	7	8

1	2	3
---	---	---